Laboratorio: agregar funcionalidades con JavaScript

Proyecto EventSales · HTML, CSS y JavaScript

Elaborado por: Henry Antonio Mendoza Puerta by hampcode

Índice

- Objetivo
- Herramientas
- Requisitos
- Parte 1. Prepara tu entorno
- Parte 2. Las cuatro pantallas
  - Pantalla 1. Landing
  - Pantalla 2. Login
  - Pantalla 3. Crear cuenta de participante
  - Pantalla 4. Eventos
- Parte 3. Cierre: merge y versión (tag)
- Lista de revisión

Objetivo

Agregar JavaScript a las cuatro pantallas que ya existen en develop. Su HTML y su CSS ya están hechos y no los cambias: solo escribes JavaScript. Por eso cada pantalla lleva una rama nueva y su propio archivo en la carpeta js. De momento los datos se guardan en un array dentro de ese archivo.

Volver al índice

Herramientas

- Visual Studio Code: editor donde abres el proyecto y la terminal.
- JavaScript: lenguaje del navegador que da comportamiento a la página: clics, mensajes y filtros. Va en su propio archivo dentro de js, nunca dentro del HTML, y se enlaza al final del body.
- Array: lista de datos dentro del archivo JavaScript. De momento los datos viven ahí y se pierden al recargar la página; más adelante se guardarán en una base de datos.
- Navegador: ahí pruebas tu página. Con F12 abres la consola, que muestra los errores de JavaScript.
- Git: control de versiones en tu equipo. Guarda tus cambios en ramas y commits.
- GitHub: repositorio en la nube donde se comparte el trabajo con Pull Requests, merge y tag de versión.

Volver al índice

Requisitos

- Visual Studio Code y un navegador con consola (F12).
- La entrega v1.0.0 de la guía anterior: las cuatro pantallas con su HTML y CSS ya están en main y develop.
- Tus pantallas asignadas, en este orden: Landing, Login, Crear cuenta y Eventos.

Volver al índice

Parte 1. Prepara tu entorno

1. Actualiza tu copia del proyecto con lo último de develop (si no lo has clonado, mira la guía anterior):

```
Terminal
git checkout develop
git pull origin develop
```

2. Abre la carpeta del proyecto en Visual Studio Code y abre la terminal (Terminal, New Terminal).

Volver al índice

Parte 2. Las cuatro pantallas

Trabajas sobre el HTML y el CSS que ya existen: no agregas HTML ni CSS, solo el script y el JavaScript. Cada pantalla tiene su rama nueva y su archivo separado en js, enlazado al final del body. Los datos se guardan en un array dentro de ese archivo.

Sintaxis moderna de JavaScript (ES6) que usarás en las cuatro pantallas:

```
Sintaxis
// const guarda valores que no cambian; let, los que sí cambian
const nombre = "Ana";
let intentos = 0;

// Función flecha: (parámetros) => resultado
const saludar = (persona) => `Hola ${persona}`;

// Interpolación de texto: ${} dentro de comillas invertidas ( ` )
// en lugar de unir con +  ->  "Hola " + persona
saludar(nombre); // "Hola Ana"

// Desestructuración: toma propiedades de un objeto
const { titulo, precio } = { titulo: "Mistura", precio: 60 };
```

Pantalla 1. Landing (index.html)

Paso 1. La pantalla ya existe en develop. Crea una rama nueva para su JavaScript:

```
Terminal

git checkout develop
git pull origin develop
git checkout -b feature/landing-js
```

Paso 2. Crea el archivo js/landing.js, separado del HTML. En el Explorador de Visual Studio Code, clic derecho en la carpeta js, New File y el nombre del archivo. Luego enlázalo en index.html, al final del body, justo antes de </body>:

```
index.html

    ...
    <script src="js/landing.js"></script>
</body>
```

Paso 3. Escribe la funcionalidad en js/landing.js. Al pulsar una categoría, «Próximos eventos» muestra solo los eventos de esa categoría. Los datos viven en un array:

```
js/landing.js

// 1. Datos: de momento viven en un array
const eventos = [
  { titulo: "Vivo x el Rock", categoria: "Música", precio: 45,
    fecha: "SÁB 14 DIC · 20:00", lugar: "Explanada Monumental" },
  { titulo: "Mistura", categoria: "Gastronomía", precio: 60,
    fecha: "VIE 20 DIC · 19:30", lugar: "Costa Verde" },
  { titulo: "Peru Game Show", categoria: "Tecnología", precio: 35,
    fecha: "SÁB 11 ENE · 21:00", lugar: "Centro de Convenciones" },
  { titulo: "Symphony Under Stars", categoria: "Teatro", precio: 55,
    fecha: "VIE 17 ENE · 19:30", lugar: "Teatro Nacional" },
  { titulo: "Maratón Lima 10K", categoria: "Deportes", precio: 25,
    fecha: "DOM 26 ENE · 07:00", lugar: "Costa Verde" },
];

// 2. Elementos de la página
const grid = document.querySelector(".upcoming__grid");
const categorias = document.querySelectorAll(".category");

// 3. Función que dibuja las tarjetas
const mostrar = (lista) => {
  grid.innerHTML = lista
    .map(({ titulo, categoria, fecha, lugar, precio }) => `
      <article class="event-card">
        <div class="event-card__image"></div>
        <p class="event-card__tag">${categoria}</p>
        <div class="event-card__info">
          <p class="event-card__date">${fecha}</p>
          <h3 class="event-card__title">${titulo}</h3>
          <p class="event-card__venue">${lugar}</p>
          <div class="event-card__footer">
            <p>
              <span class="event-card__price-label">Desde</span>
              <span class="event-card__price">S/ ${precio}</span>
```

```
            </p>
            <a class="btn btn--secondary"
               href="pages/participante/event-detail.html">Ver entradas</a>
          </div>
        </div>
      </article>
    `)
    .join("");
};

// 4. Evento: al pulsar una categoría
categorias.forEach((categoria) => {
  categoria.addEventListener("click", (clic) => {
    clic.preventDefault();

    const nombre = categoria.querySelector(".category__name").textContent;

    categorias.forEach((c) => c.classList.remove("category--active"));
    categoria.classList.add("category--active");

    mostrar(eventos.filter((evento) => evento.categoria === nombre));
  });
});
```

Practicas: array de objetos, funciones flecha, desestructuración, interpolación de texto, map, filter y classList.

Paso 4. Prueba en el navegador. Pulsa una categoría, por ejemplo Teatro: cambia la lista de próximos eventos. Si algo falla, abre la consola con F12.

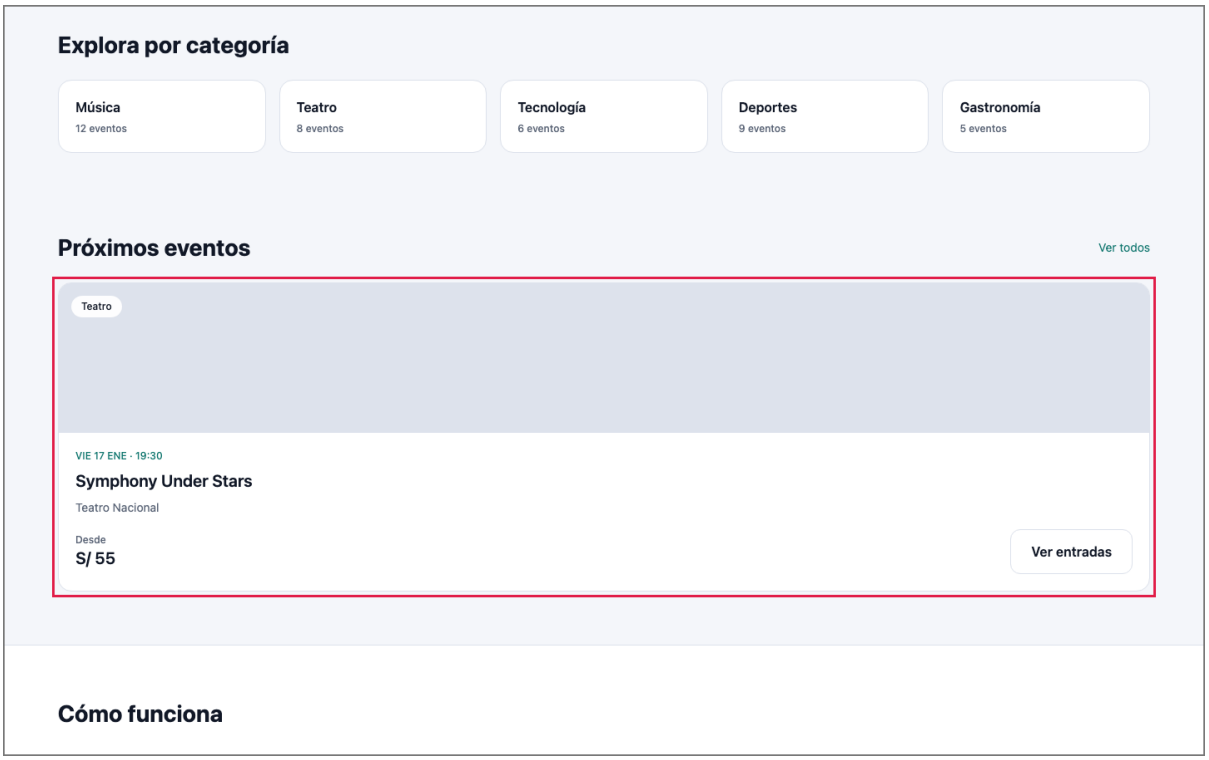


Figura 1. Al pulsar Teatro, «Próximos eventos» solo muestra los de esa categoría.

Paso 5. Guarda tu trabajo y sube la rama:

```
Terminal

git add .
git commit -m "feat(landing): filtra los próximos eventos por categoría"
git push -u origin feature/landing-js
```

Paso 6. Abre el Pull Request de feature/landing-js hacia develop. En GitHub, tras el push, pulsa Compare & pull request:

```
Pull Request en GitHub

Base: develop
Compare: feature/landing-js
Título: feat(landing): filtra los próximos eventos por categoría
Descripción: Agrega js/landing.js: las categorías filtran los eventos de un
array.
```

Recomendación: pide a un compañero que revise el PR y no hagas merge del tuyo. Revisa en Files changed que solo estén el archivo JavaScript y el script de tu pantalla.

Volver al índice

Pantalla 2. Login (pages/auth/login.html)

Paso 1. La pantalla ya existe en develop. Crea una rama nueva para su JavaScript:

```
Terminal

git checkout develop
git pull origin develop
git checkout -b feature/login-js
```

Paso 2. Crea el archivo js/login.js, separado del HTML. En el Explorador de Visual Studio Code, clic derecho en la carpeta js, New File y el nombre del archivo. Luego enlázalo en pages/auth/login.html, al final del body, justo antes de </body>:

```
pages/auth/login.html

    ...
    <script src="../../js/login.js"></script>
</body>
```

Paso 3. Escribe la funcionalidad en js/login.js. Al pulsar Iniciar sesión, valida el correo y la contraseña; si no coinciden, muestra un mensaje con los intentos fallidos en el texto de introducción. Los datos viven en un array:

```
js/login.js

// 1. Datos: de momento los usuarios viven en un array
const usuarios = [
  { correo: "ana@correo.com", clave: "12345678" },
```

```
  { correo: "luis@correo.com", clave: "abcdefgh" },
];

// 2. Elementos de la página
const correo = document.querySelector("#email");
const clave = document.querySelector("#password");
const mensaje = document.querySelector(".auth-form__intro");
const boton = document.querySelector(".auth-form__submit");

// 3. Estado: este valor cambia, por eso usa let
let intentos = 0;

// 4. Evento: al pulsar Iniciar sesión
boton.addEventListener("click", (clic) => {
  clic.preventDefault();

  const existe = usuarios.find(
    (usuario) =>
      usuario.correo === correo.value && usuario.clave === clave.value
  );

  if (existe) {
    window.location.href = boton.href;
  } else {
    intentos++;
    mensaje.textContent = `Datos incorrectos. Intentos: ${intentos}`;
    mensaje.style.color = "#b91c1c";
  }
});
```

Practicas: let para el contador de intentos, find, if/else, interpolación de texto y preventDefault.

Paso 4. Prueba en el navegador. Escribe un correo y una contraseña que no estén en el array: aparece el mensaje. Con ana@correo.com y 12345678 entra a Eventos. Si algo falla, abre la consola con F12.

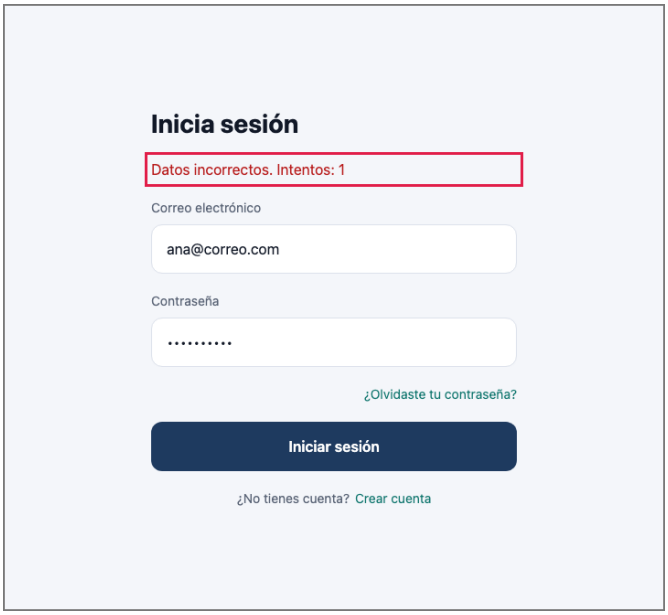


Figura 2. Mensaje con el número de intentos fallidos.

Paso 5. Guarda tu trabajo y sube la rama:

```
Terminal

git add .
git commit -m "feat(login): valida el acceso con un array de usuarios"
git push -u origin feature/login-js
```

Paso 6. Abre el Pull Request de feature/login-js hacia develop. En GitHub, tras el push, pulsa Compare & pull request:

```
Pull Request en GitHub

Base: develop
Compare: feature/login-js
Título: feat(login): valida el acceso con un array de usuarios
Descripción: Agrega js/login.js: valida correo y contraseña contra un array.
```

Recomendación: pide a un compañero que revise el PR y no hagas merge del tuyo. Revisa en Files changed que solo estén el archivo JavaScript y el script de tu pantalla.

Volver al índice

Pantalla 3. Crear cuenta de participante (pages/auth/signup.html)

Paso 1. La pantalla ya existe en develop. Crea una rama nueva para su JavaScript:

```
Terminal

git checkout develop
git pull origin develop
git checkout -b feature/signup-js
```

Paso 2. Crea el archivo js/signup.js, separado del HTML. En el Explorador de Visual Studio Code, clic derecho en la carpeta js, New File y el nombre del archivo. Luego enlázalo en pages/auth/signup.html, al final del body, justo antes de </body>:

```
pages/auth/signup.html

    ...
    <script src="../../js/signup.js"></script>
</body>
```

Paso 3. Escribe la funcionalidad en js/signup.js. Al pulsar Crear cuenta, valida los campos y guarda al usuario en el array. Los datos viven en un array:

```
js/signup.js

// 1. Datos: de momento los usuarios viven en un array
const usuarios = [];

// 2. Elementos de la página
const nombre = document.querySelector("#name");
const correo = document.querySelector("#email");
```

```
const clave = document.querySelector("#password");
const mensaje = document.querySelector(".auth-form__intro");
const boton = document.querySelector(".auth-form__submit");

// 3. Evento: al pulsar Crear cuenta
boton.addEventListener("click", (clic) => {
  clic.preventDefault();

  const incompleto =
    !nombre.value || !correo.value || clave.value.length < 8;

  if (incompleto) {
    mensaje.textContent = "Completa todo. La contraseña lleva 8 o más.";
    mensaje.style.color = "#b91c1c";
    return;
  }

  usuarios.push({
    nombre: nombre.value,
    correo: correo.value,
    clave: clave.value,
  });

  const total = usuarios.length;
  mensaje.textContent = `Cuenta creada. Usuarios registrados: ${total}`;
  mensaje.style.color = "#0f766e";
});
```

Practicas: array vacío, push, validación con if e interpolación de texto.

Paso 4. Prueba en el navegador. Pulsa Crear cuenta con campos vacíos: aparece el aviso. Con todo completo, el mensaje cuenta los usuarios del array. Si algo falla, abre la consola con F12.

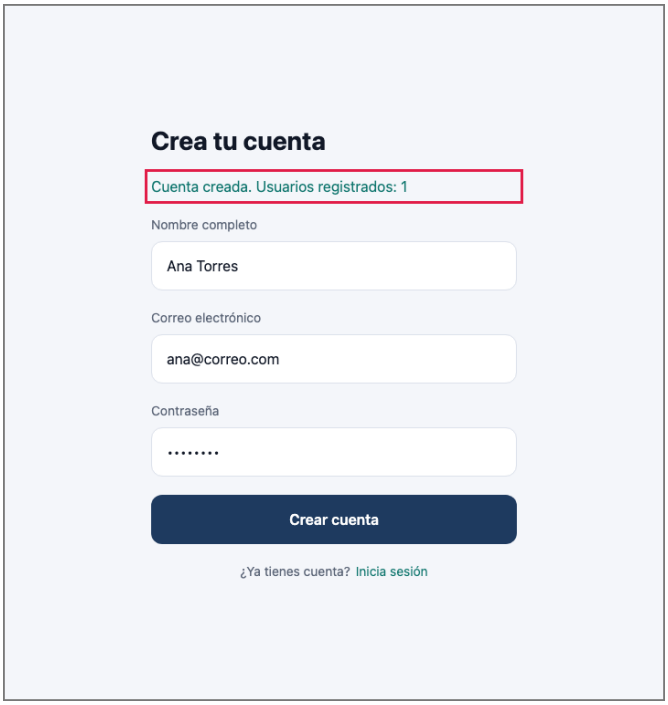


Figura 3. Mensaje tras crear una cuenta.

Paso 5. Guarda tu trabajo y sube la rama:

```
Terminal

git add .
git commit -m "feat(signup): registra usuarios en un array"
git push -u origin feature/signup-js
```

Paso 6. Abre el Pull Request de feature/signup-js hacia develop. En GitHub, tras el push, pulsa Compare & pull request:

```
Pull Request en GitHub

Base: develop
Compare: feature/signup-js
Título: feat(signup): registra usuarios en un array
Descripción: Agrega js/signup.js: valida los campos y guarda el usuario en un
array.
```

Recomendación: pide a un compañero que revise el PR y no hagas merge del tuyo. Revisa en Files changed que solo estén el archivo JavaScript y el script de tu pantalla.

Volver al índice

Pantalla 4. Eventos (pages/participante/events.html)

Paso 1. La pantalla ya existe en develop. Crea una rama nueva para su JavaScript:

```
Terminal

git checkout develop
git pull origin develop
git checkout -b feature/eventos-js
```

Paso 2. Crea el archivo js/events.js, separado del HTML. En el Explorador de Visual Studio Code, clic derecho en la carpeta js, New File y el nombre del archivo. Luego enlázalo en pages/participante/events.html, al final del body, justo antes de </body>:

```
pages/participante/events.html

    ...
    <script src="../../js/events.js"></script>
</body>
```

Paso 3. Escribe la funcionalidad en js/events.js. La lista sale del array y se filtra con los chips de categoría y con el buscador. Los datos viven en un array:

```
js/events.js

// 1. Datos: de momento viven en un array
const eventos = [
  { titulo: "Neon Nights Festival", categoria: "Música", precio: 45,
    fecha: "SÁB 14 DIC · 20:00", lugar: "Arena Central" },
  { titulo: "Symphony Under Stars", categoria: "Teatro", precio: 60,
```

```
    fecha: "VIE 20 DIC · 19:30", lugar: "Teatro Nacional" },
  { titulo: "Tech & Beats Live", categoria: "Tecnología", precio: 35,
    fecha: "SÁB 11 ENE · 21:00", lugar: "Club Aurora" },
  { titulo: "Rock en el Parque", categoria: "Música", precio: 40,
    fecha: "DOM 19 ENE · 16:00", lugar: "Parque de la Exposición" },
  { titulo: "Maratón Lima 10K", categoria: "Deportes", precio: 25,
    fecha: "DOM 26 ENE · 07:00", lugar: "Costa Verde" },
];

// 2. Elementos de la página
const grid = document.querySelector(".events__grid");
const chips = document.querySelectorAll(".events__chip");
const buscador = document.querySelector(".events__search-input");
const subtitulo = document.querySelector(".events__subtitle");

// 3. Estado: la categoría elegida cambia, por eso usa let
let categoriaActual = "Todos";

// 4. Función que filtra y dibuja las tarjetas
const mostrar = () => {
  const texto = buscador.value.toLowerCase();

  const lista = eventos.filter(({ titulo, categoria }) => {
    const coincideCategoria =
      categoriaActual === "Todos" || categoria === categoriaActual;
    const coincideTexto = titulo.toLowerCase().includes(texto);

    return coincideCategoria && coincideTexto;
  });

  grid.innerHTML = lista
    .map(({ titulo, categoria, fecha, lugar, precio }) => `
      <li class="events__item">
        <article class="event-card">
          <div class="event-card__image"></div>
          <span class="event-card__tag">${categoria}</span>
          <div class="event-card__info">
            <p class="event-card__date">${fecha}</p>
            <h2 class="event-card__title">${titulo}</h2>
            <p class="event-card__venue">${lugar}</p>
            <div class="event-card__footer">
              <p>
                <span class="event-card__price-label">Desde</span>
                <span class="event-card__price">S/ ${precio}</span>
              </p>
              <a class="btn btn--secondary"
                 href="event-detail.html">Ver entradas</a>
            </div>
          </div>
        </article>
      </li>
    `)
    .join("");

  subtitulo.textContent = `${lista.length} eventos disponibles`;
};

// 5. Eventos: chips de categoría y buscador
chips.forEach((chip) => {
  chip.addEventListener("click", (clic) => {
    clic.preventDefault();
    categoriaActual = chip.textContent;
```

```
    chips.forEach((c) => c.classList.remove("events__chip--active"));
    chip.classList.add("events__chip--active");
    mostrar();
  });
});

buscador.addEventListener("input", mostrar);

// 6. Al cargar la página, dibuja todos los eventos
mostrar();
```

Practicas: let para el estado, filter con dos condiciones, desestructuración, map y join, y el evento input.

Paso 4. Prueba en el navegador. Pulsa los chips de categoría y escribe en el buscador: la lista y el texto «eventos disponibles» cambian. Si algo falla, abre la consola con F12.

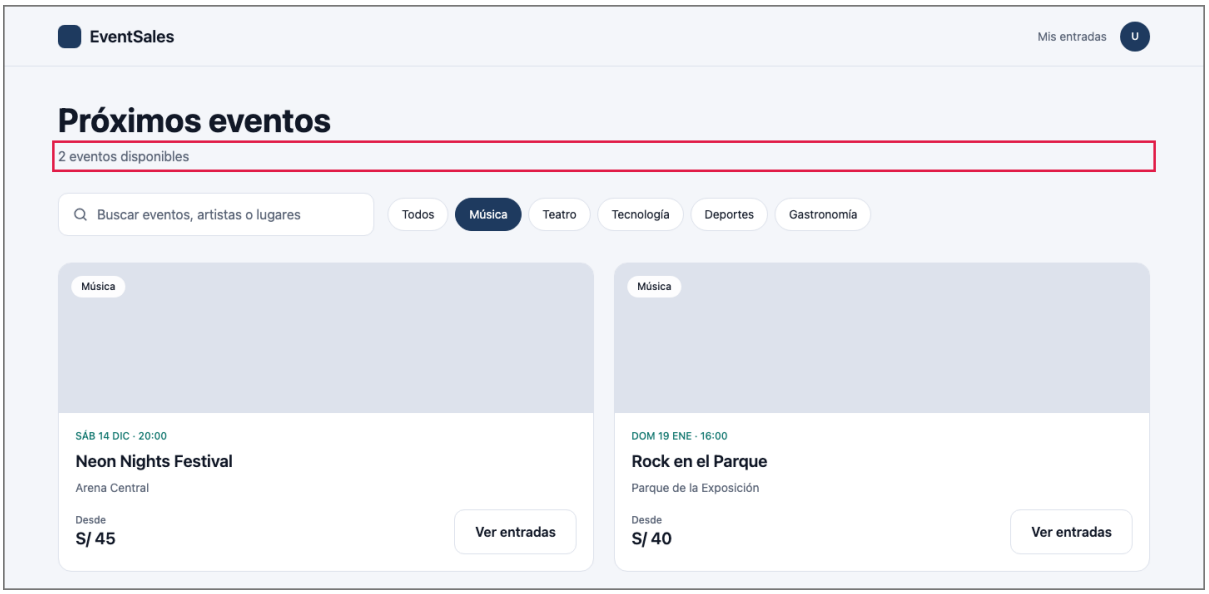


Figura 4. Al elegir Música, quedan dos eventos.

Paso 5. Guarda tu trabajo y sube la rama:

```
Terminal

git add .
git commit -m "feat(eventos): lista y filtra los eventos desde un array"
git push -u origin feature/eventos-js
```

Paso 6. Abre el Pull Request de feature/eventos-js hacia develop. En GitHub, tras el push, pulsa Compare & pull request:

```
Pull Request en GitHub

Base: develop
Compare: feature/eventos-js
Título: feat(eventos): lista y filtra los eventos desde un array
Descripción: Agrega js/events.js: los eventos salen de un array y se filtran
por categoría y búsqueda.
```

Recomendación: pide a un compañero que revise el PR y no hagas merge del tuyo. Revisa en Files changed que solo estén el archivo JavaScript y el script de tu pantalla.

Volver al índice

Parte 3. Cierre: merge y versión (tag)

1. Cuando tu Pull Request se une a develop, actualiza tu equipo y borra las ramas que ya no usas:

```
Terminal

git checkout develop
git pull origin develop
git branch -d feature/landing-js feature/login-js feature/signup-js
feature/eventos-js
```

2. Cuando las cuatro pantallas están en develop y funcionan, el líder abre un Pull Request de develop hacia main:

```
Pull Request en GitHub

Base: main
Compare: develop
Título: release: funcionalidades con JavaScript
Descripción: Landing, Login, Crear cuenta y Eventos con JavaScript.
```

Únelo con Create a merge commit (no squash) para conservar el historial de cada pantalla.

3. Crea la nueva versión (tag) sobre main:

```
Terminal

git checkout main
git pull origin main
git tag -a v1.1.0 -m "Funcionalidades con JavaScript: Landing, Login, Crear
cuenta y Eventos"
git push origin v1.1.0
git tag
```

Agregaste funcionalidades nuevas sin romper lo anterior, así que sube el número MENOR: de v1.0.0 a v1.1.0. Si solo corriges un error, sube el PARCHE (v1.1.1).

Volver al índice

Lista de revisión

- Cada pantalla está en su propia rama feature y con su Pull Request hacia develop, revisado por un compañero.
- Cada pantalla tiene su archivo en js, separado del HTML y enlazado al final del body, antes de </body>.
- No agregaste HTML ni CSS: solo el script y el JavaScript.
- Los datos están en un array dentro de cada archivo JavaScript.
- Probaste cada funcionalidad en el navegador y la consola (F12) no muestra errores.
- Después del merge actualizaste develop y creaste el tag v1.1.0 en main.

Volver al índice